Dataplatr

Agentic ELT Solution for Financial Analytics Using Snowflake Cortex AI

Executive Overview

The **Financial Analytics (Sage Intacct - General Ledger)** Data Share bridges the gap between complex, siloed accounting records and actionable business intelligence. Built natively on the Snowflake Data Cloud, this offering provides high-fidelity, granular data models that translate raw Sage Intacct ledger logic into a structured format optimized for immediate analysis.

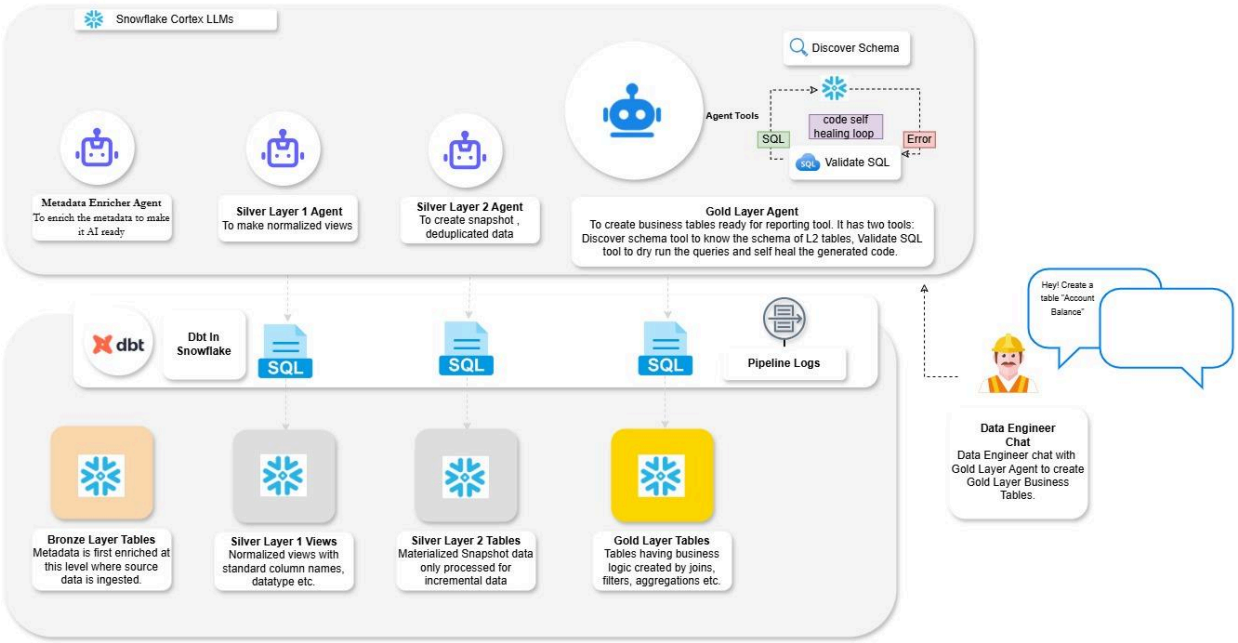
By delivering **pre-built, analytics-ready General Ledger (GL) schemas** and an AI based ***no-code*** solution for developing data pipelines, it empowers finance and analytics teams to deploy sophisticated **ELT pipelines and workflows** using the familiar tech stack of snowflake and dbt (in-house) without the burden of manual data engineering. This solution fundamentally transforms fragmented financial tables into a unified data foundation, topping it with an a modular, agent-based approach to:

1. **Enrich** Bronze data with metadata (L0).
2. **Generate** standardized SQL for Silver views (L1).
3. **Generate** incremental Dbt SQL for materialized tables (L2).
4. **Empower** users with a conversational agent to build and deploy validated Gold tables (L3) as per business requirements.

The agentic ELT pipeline uses **self-healing**, SQL validation and **Human-In-The-Loop** concepts to retain reliability and accountability in the process.

Dataplatr

Agentic ELT Pipeline : Architecture Overview



The entire pipeline follows the standard Medallion Architecture (Bronze -> Silver -> Gold), augmented by intelligent agents at each step.

The flow is as follows:

Bronze (L0): Raw data is ingested.

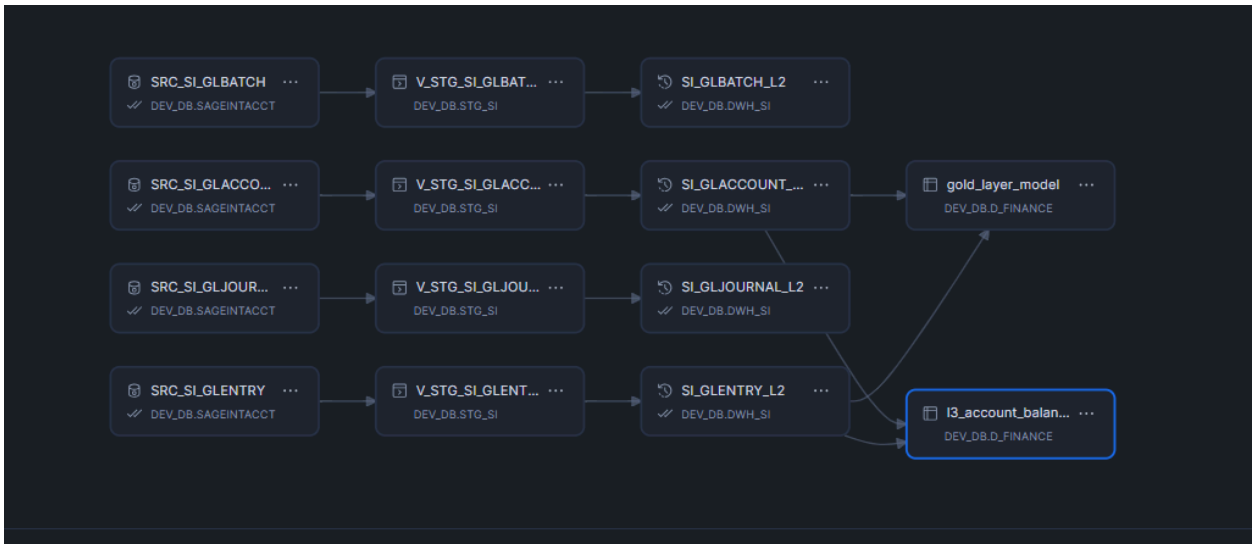
Metadata Enricher: An agent scans Bronze tables and adds table/column comments using a human-in-the-loop (HITL) process.

Silver (L1): An agent generates SQL to create standardized views (e.g., proper column names) which are executed using dbt (in-house).

Silver (L2): An agent assists a human in defining incremental logic and generates dbt models.

Gold (L3): An advanced conversational agent (with robust validation and self-healing) chats with an engineer to build business-ready tables, also outputting dbt SQL model.

Dbt in Snowflake : The execution of the models, quality checks and lineage mapping is handled by in-house dbt.



Construction & Data Strategy

This solution is built on the Snowflake Data Cloud, providing an intelligent yet reliable solution. Using the four foundational pillars—**GLACCOUNT, GLENTRY, GLBATCH,** and **GLJOURNAL**—it establishes a strong foundation for agent-driven finance analytics workflows.

Data Model Summary

Table	Description	Records
SI_GLENTRY_L2	GL transaction entries with amounts, dates, locations	245,909
SI_GLACCOUNT_L2	Chart of accounts master data	450
SI_GLBATCH_L2	Journal batch headers	6,021
SI_GLJOURNAL_L2	Journal definitions	25

Key Features

Agent-Driven Automation (Reduced Manual Effort)

- Automates end-to-end ELT workflows including table onboarding, SQL generation, and pipeline orchestration
- Minimizes manual coding through metadata-driven and agent-assisted model creation
- Accelerates development cycles and reduces dependency on most of the real-time engineering tasks

Built-in Validations & Data Trust

- Incorporates multi-layer validations across Bronze, Silver, and Gold to ensure data quality
- Schema checks, null handling, and transformation-level validations improve reliability
- Ensures consistent and trustworthy outputs for downstream analytics and reporting

Human-in-the-Loop Governance

- Critical steps (e.g., SQL generation, table approvals) include user validation checkpoints
- Enables controlled automation with the ability to review, approve, or override decisions
- Balances speed with governance, ensuring production-grade confidence

Reliable & Resilient Pipelines

- Designed for idempotent and repeatable builds to avoid data duplication or inconsistencies
- Handles incremental processing efficiently for large-scale datasets
- Integrates seamlessly with CI/CD (Github) workflows for consistent deployments

Lineage-Aware Transformations

- Tracks dependencies across models and layers for full data lineage visibility
- Enables traceability from source (Bronze) to business-ready outputs (Gold)
- Simplifies impact analysis, debugging, and auditability

Scalable & Domain-Agnostic Framework

- Easily extensible to multiple domains such as Finance (Sage Intacct), HR (Workday), or ERP (Oracle)
- Modular design supports rapid onboarding of new data sources and use cases
- Built to scale with growing data volumes and complexity

Business Value

- ⏱️ ***Faster Time-to-Insight***
- 💸 ***Reduced Engineering Costs***

- 📈 ***Improved Data Reliability & Trust***
- 🔍 ***Enhanced Transparency & Governance***
- 🚀 ***Scalable Across Domains***
- 🔄 ***Increased Operational Efficiency***
- 🤝 ***Business–Tech Alignment***

The Dataplatr Advantage

The Dataplatr **Agentic ELT Solution for Financial Analytics** provides an agentic way of deploying your data pipelines. By combining Agentic ELT automation with Dataplatr's deep ***expertise in financial customization***, we help enterprises modernize ERP workflows and achieve faster time-to-insight.

Contact us at: **info@dataplatr.com**

For consultations or custom inquiries: https://dataplatr.com/contact-us

Linkedin

Set Up Instructions:

Architecture Overview

- **Frontend:** Streamlit (containerized)

- **Runtime:** Snowpark Container Services

- **Security Model:** Snowflake Native App (consumer-controlled permissions)

- **External Connectivity:** External Access Integration (optional, approval required)

All compute, storage, and execution occur **within the consumer's Snowflake account**.

Required Setup (Consumer Account)

Before installing the application, the following objects must exist in the consumer account. These steps are required **only if external access is enabled**.

1. Network Rule

Create a network rule allowing outbound access to the required external services:

```sql
CREATE NETWORK RULE ai_elt_network_rule
  TYPE = HOST_PORT
  MODE = EGRESS
  VALUE_LIST = (
    'api.github.com:443',
    'raw.githubusercontent.com:443',
    'pypi.org:443',
    'files.pythonhosted.org:443',
    'aiplatform.googleapis.com:443',
    'us-central1-aiplatform.googleapis.com:443',
    'generativelanguage.googleapis.com:443',
    'oauth2.googleapis.com:443',
    'sts.googleapis.com:443',
    'iamcredentials.googleapis.com:443'
  );
```

2. Create an External Access Integration

Create an External Access Integration that references the network rule:

```sql

CREATE EXTERNAL ACCESS INTEGRATION AI_ELT_ACCESS

  ALLOWED_NETWORK_RULES = (ai_elt_network_rule)

  ENABLED = TRUE;

```

⚠️ This integration must be approved by a Snowflake account administrator during or after installation.

3. Create Required Secrets

Create the required secrets in the consumer account:

```sql

CREATE SECRET github_pat

  TYPE = GENERIC_STRING

  SECRET_STRING = '<your_github_personal_access_token>';


CREATE SECRET vertex_sa

  TYPE = GENERIC_STRING

  SECRET_STRING = '<your_vertex_ai_service_account_json>';

```

Dataplatr

Installing the Application

Install the application using the following SQL command:

```sql
CREATE APPLICATION ai_elt_app
  FROM APPLICATION PACKAGE AI_ELT_NATIVE_APP
  USING
    external_access_integrations = (AI_ELT_ACCESS)
    secrets = (
      github_pat = github_pat,
      vertex_sa = vertex_sa
    )
    compute_pool = my_pool;
```

The compute pool must already exist and be available to the installing role.

Consumer Database/Schema Access (Required)

After installation, the consumer must grant caller privileges to the app and then set the consumer DB/schema configuration.

1. Grant caller privileges to the application:

```sql
-- Required: caller usage on consumer database
GRANT CALLER USAGE ON DATABASE <consumer_db> TO APPLICATION <app_name>;

-- Recommended: inherited usage on all schemas in that database
GRANT INHERITED CALLER USAGE ON ALL SCHEMAS IN DATABASE <consumer_db>
  TO APPLICATION <app_name>;

-- Required for read flows
GRANT INHERITED CALLER SELECT ON ALL TABLES IN DATABASE <consumer_db>
  TO APPLICATION <app_name>;

-- Optional (only if app writes to consumer tables)
GRANT INHERITED CALLER INSERT ON ALL TABLES IN DATABASE <consumer_db>
  TO APPLICATION <app_name>;
GRANT INHERITED CALLER UPDATE ON ALL TABLES IN DATABASE <consumer_db>
  TO APPLICATION <app_name>;
GRANT INHERITED CALLER DELETE ON ALL TABLES IN DATABASE <consumer_db>
  TO APPLICATION <app_name>;

-- Optional (if app creates objects in consumer schemas)
GRANT INHERITED CALLER CREATE TABLE, CREATE VIEW, CREATE STAGE
```

```
  ON ALL SCHEMAS IN DATABASE <consumer_db>

  TO APPLICATION <app_name>;
```

2. Configure the app to use the consumer-selected database/schema:

```sql
CALL <app_name>.CONFIG.SET_CONSUMER_DB_SCHEMA('<consumer_db>', '<consumer_schema>');
```

3. Optional verification:

```sql
CALL <app_name>.CONFIG.GET_CONSUMER_CONFIG();
```

Notes:

- `SET_CONSUMER_DB_SCHEMA` runs with restricted caller's rights and validates access using caller privileges.

- App-internal config is stored by owner-rights helper procedures in `APP_PRIVATE.CONSUMER_CONFIG`.

External Services Used

This application will integrate with the following external services **only after customer approval**:

- **Google Vertex AI** – Large Language Model inference

- **GitHub APIs** – Versioned artifact access

- **PyPI** – Python package resolution